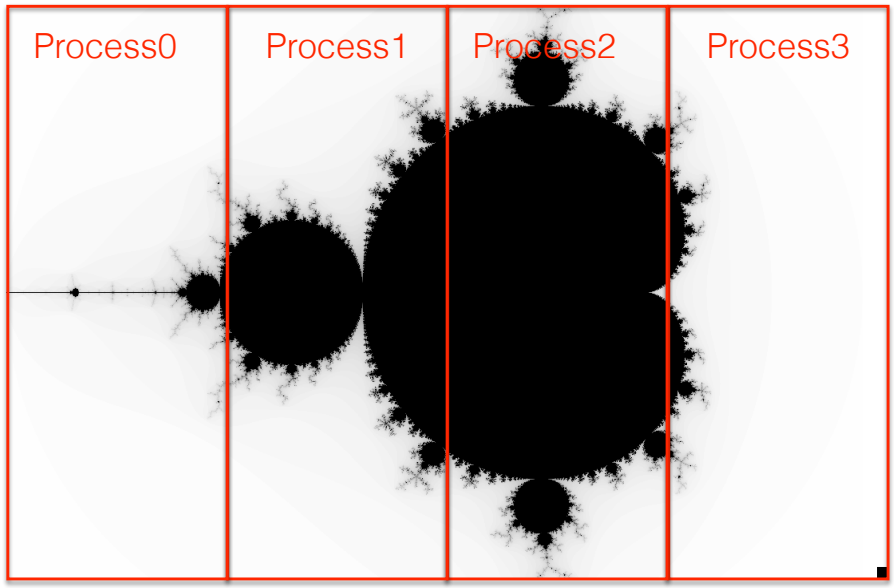
Domain decomposition

Domain decomposition

- With MPI we have to distribute the work between MPI processes (no equivalent of `omp parallel for`)
- One common method is called “domain decomposition”
- Domain decomposition can be used with PDE solvers
- We will use domain decomposition as an example of how MPI can be used

Domain decomposition

- Distribute work by dividing the computational domain over different MPI processes



Using MPI collectives for PDE solver domain decomposition

- Rank zero process reads input parameters from a file and sends to all processes (broadcast)
- Rank zero process reads initial conditions from a file and sends to all processes (scatter)
- Rank zero process collects results from multiple processes and writes output to a file (gather)

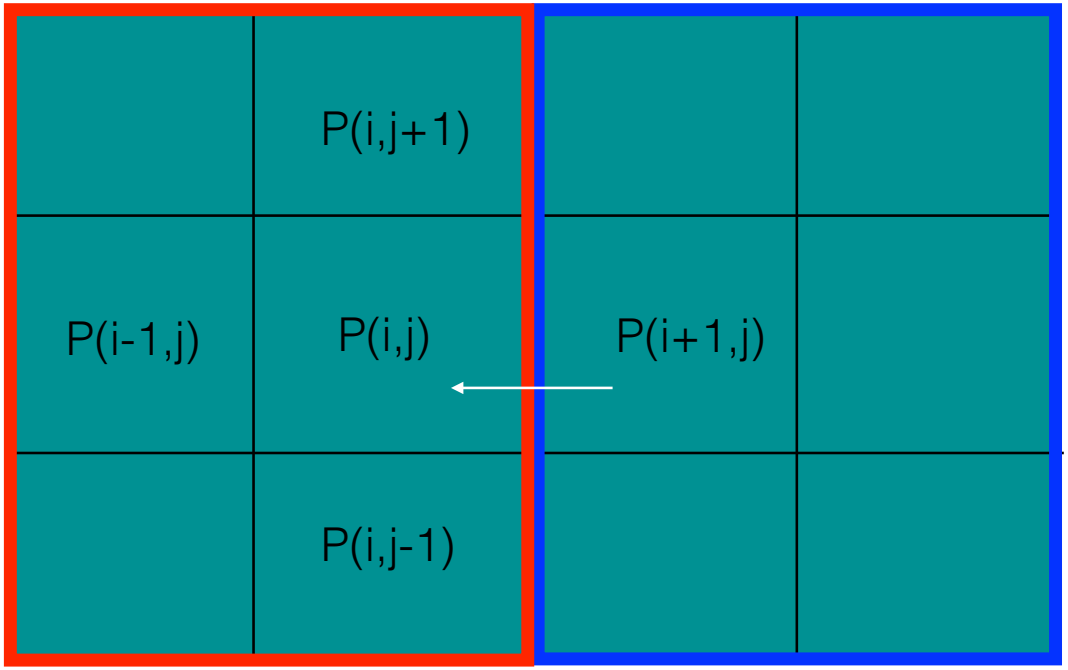
Time step in PDE solver

- All MPI processes need to use the same time step when updating their solution to the PDE
- If the time step is determined by a stability constraint (e.g. Courant condition) and the velocity varies then different parts of the domain will have a different maximum allowable time step
- The time step needs to be the minimum value from the whole computational domain
- All MPI processes pass their minimum time step to `MPI_Allreduce` with an `MPI_MIN` operator
- The result will be the minimum time step across all MPI processes

Finite difference stencils and domain decomposition

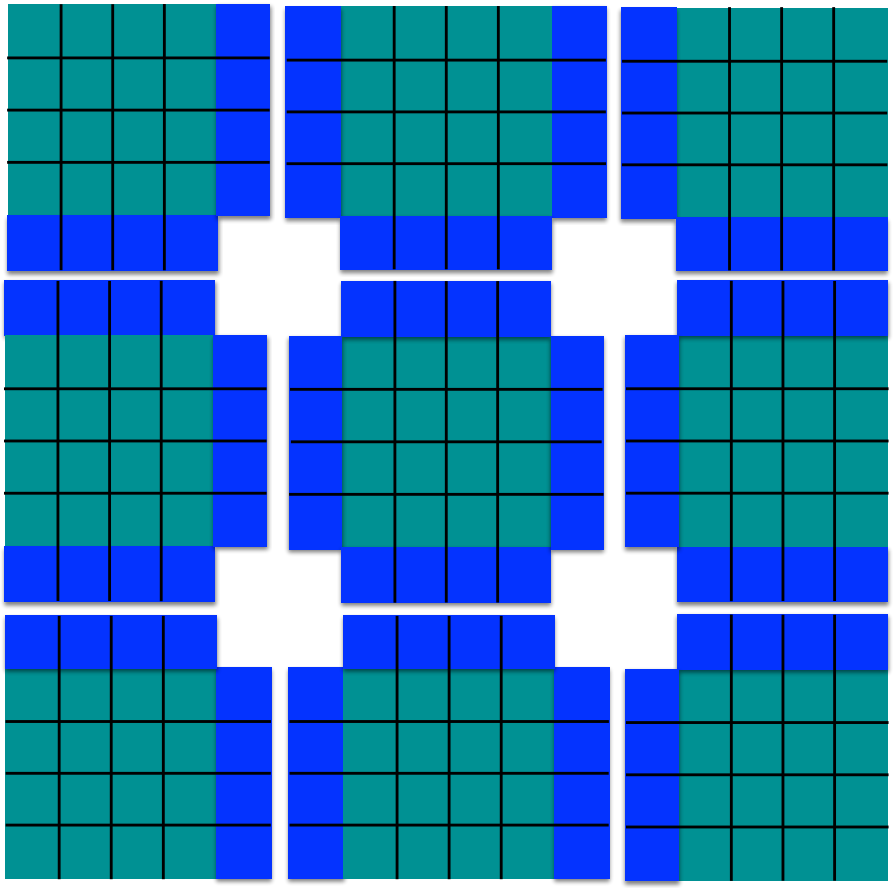
- When calculating a finite difference stencil we need values from neighbouring grid points
- What happens if we have decomposed the domain?
- We will need to communicate at values at domain boundaries where the stencil extends into the neighbouring domain

Finite difference stencils and domain decomposition



Could communicate information on demand or store a “halo” (copy of row/column held by another MPI process).

Finite difference stencils and domain decomposition



3x3 decomposition of 12x12 2D domain

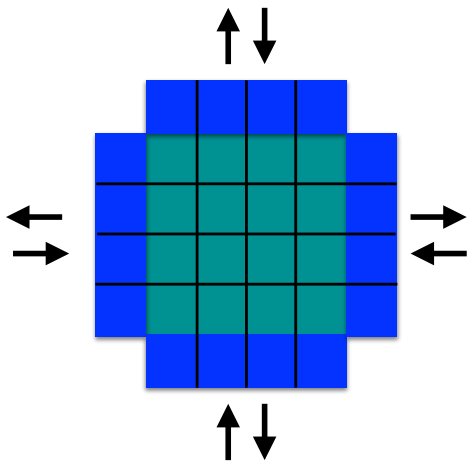
Blue regions are haloes

Finite difference stencils and domain decomposition

```
MPI_Isend(left)
MPI_Irecv(left)

MPI_Isend(right)
MPI_Irecv(right)

MPI_Isend(up)
MPI_Irecv(up)

MPI_Isend(down)
MPI_Irecv(down)

MPI_Waitall
```

Haloes need to be updated when the solution is updated

Send and receive halos with non-blocking point-to-point communications



Unit summary

- Domain decomposition is a common method for distributing a calculation between MPI processes
- MPI collectives can be used when working with the whole domain or global values
- Calculating a finite difference stencil requires values from neighbouring domains
- A “halo” of values from neighbouring domains can be stored
- Neighbouring domains need to exchange haloes when the solution is updated
- Non-blocking point-to-point communication can be used for halo exchange